

1 fast-layout

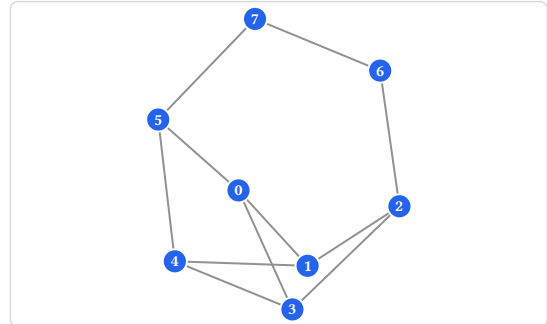
Version 0.1.0

fast-layout computes graph coordinates. CeTZ turns those coordinates into lines and nodes. Keeping these jobs separate lets the same layout drive labels, custom shapes, or another drawing package.

Node indices start at zero. Position *i* belongs to node *i*, including isolated nodes.

1.1 First layout

```
#let result = layout(  
  8, edges,  
  algorithm: "stress",  
  seed: 7,  
)
```

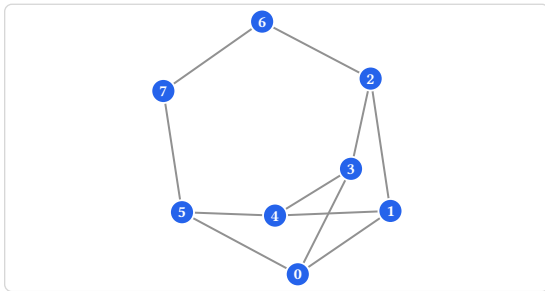


The default stress method is SGD with at most 15 updates. The result contains positions, iterations, converged, and objective.

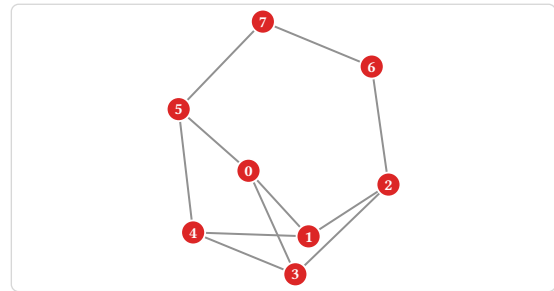
1.2 Choose a stress method

Majorization uses constrained global solves and rejects updates that increase stress beyond roundoff. SGD visits shuffled node pairs with a decaying step schedule. It can reach a lower objective on some graphs, but it does not enforce a monotonic global objective. Fifteen SGD updates are a useful low-cost recipe.

Majorization (100-update automatic budget)



Default SGD (15 updates)



```
#let major = layout(  
  8, edges,  
  algorithm: "stress",  
  stress-method: "majorization",  
)
```

Omitting iterations gives SGD 15 updates and majorization 100. An explicit value overrides either automatic budget.

Both methods use edge weights as target lengths. A weight of 2 asks for twice the path length of a weight of 1.

1.3 The five algorithms

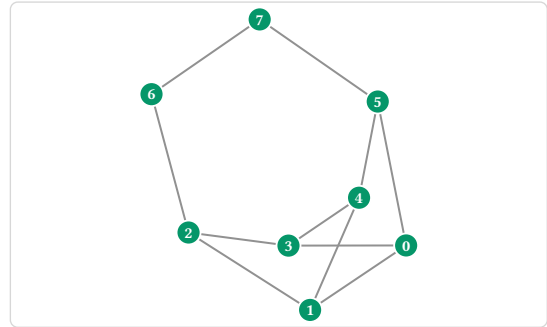
1.3.1 Stress

Stress fits Euclidean distances to weighted shortest-path distances. It accepts initial coordinates and per-coordinate pins. Majorization and SGD optimize the same objective with different update rules.

1.3.2 Spring

Spring uses Fruchterman-Reingold attraction and repulsion. Edge weights do not change its force. Edges only state connectivity. For 2D and 3D graphs of at least 256 nodes, the default switches to Barnes-Hut approximation. Set `theta: 0` for exact forces.

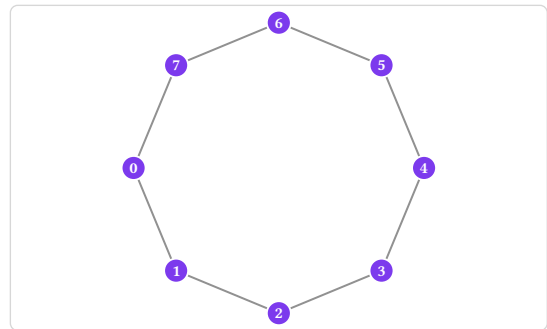
```
#layout(  
  8, edges,  
  algorithm: "spring",  
  iterations: 180,  
  seed: 7,  
)
```



1.3.3 Spectral

Spectral treats edge weights as affinities. Larger values pull endpoints into the same low-frequency modes. Optional node weights also act as affinities. Eigenvector signs can flip, so compare distances rather than raw coordinates across engines.

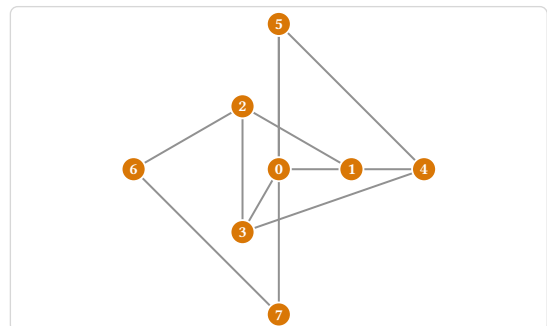
```
#layout(  
  8, cycle-edges,  
  algorithm: "spectral",  
)
```



1.3.4 Shell

Shell places explicit groups on circles, from inner to outer. Any unlisted nodes form the last shell. The alias "circular" selects the same algorithm.

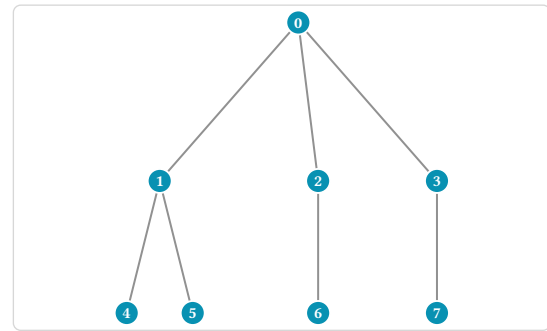
```
#layout(  
  8, edges,  
  algorithm: "shell",  
  shells: ((0,), (1, 2, 3)),  
)
```



1.3.5 Buchheim

Buchheim lays out a rooted, ordered tree. Edges point from parent to child, and input order sets sibling order. `node-sizes` contains diameters; omitted entries use 1.

```
#layout(  
  8, tree-edges,  
  algorithm: "buchheim",  
  node-sizes: (1.0, 1.4, 0.8),  
)
```



1.4 Initial positions and pins

Stress and spring accept initial positions. Use `none` for a node that should keep its seeded start. A pin mask fixes each coordinate independently.

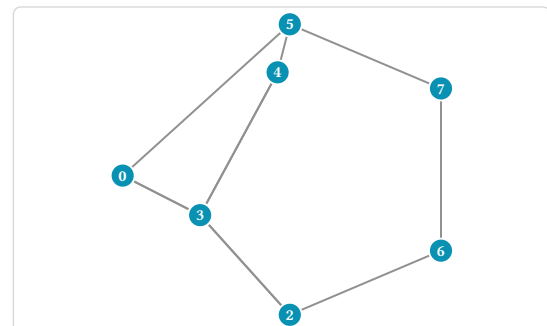
```
#let result = layout(  
  3, ((0, 1), (1, 2)),  
  initial: ((0, 0), (2, 1), none),  
  pins: ((true, true), (true, false)),  
  edge-weights: (2, 1),  
)
```

Here node 0 cannot move. Node 1 keeps its x coordinate but may move along y. A pinned coordinate without an explicit initial value stays at its seeded value.

1.5 Dimensions and projection

Stress, spring, and spectral return `dim` coordinates per node. The default is 2, and the accepted range is 1 through 4096. Shell and Buchheim are always 2D.

```
#let result = layout(  
  8, edges,  
  algorithm: "spectral",  
  dim: 3,  
)  
#let project = p => (  
  p.at(0) - 0.45 * p.at(2),  
  p.at(1) - 0.30 * p.at(2),  
)
```



The projection belongs to the drawing code. The returned positions still contain all three coordinates.

1.6 Main options

Option	Default	Meaning
algorithm	"stress"	Layout algorithm
stress-method	"sgd"	majorization or sgd; stress only
dim	2	Coordinates per node
seed	1	Initialization, collision handling, and SGD pair order
iterations	none	15 for stress SGD; 100 for other numerical methods
tolerance	1e-5	Stopping threshold
initial, pins	()	Starts and coordinate masks for stress or spring
edge-weights	()	Stress target lengths or spectral affinities
theta, c, temperature	none	Spring approximation, spacing, and movement cap
node-weights	()	Spectral node affinities
shells	()	Inner-to-outer node groups
node-sizes, root	(), 0	Buchheim diameters and root

1.7 Results and stopping

iterations counts numerical updates after initialization. converged says whether the method met its tolerance before the limit. SGD may stop after its schedule no longer clips pair steps and the largest pair-endpoint movement meets tolerance. With tolerance: 0, runs with movable coordinates use the requested budget and report converged: false. Trivial numerical layouts and fully pinned stress layouts converge without using the budget.

Stress reports weighted stress. Spring reports exact spring energy for exact-force runs and none for Barnes-Hut runs. The other algorithms report none.

For stress and spring, tolerance: 0 disables early stopping. Sparse spectral iteration checks eigenpair residuals. Shell, Buchheim, and trivial numerical layouts complete without iterative updates.

All requests are deterministic for a fixed seed and engine build. Floating-point results can differ slightly across platforms.

1.8 Drawing with CeTZ

CeTZ is a document dependency, not part of the layout engine. Pass positions into `cetz.canvas` and use a single scale for both axes.

```
#import "@preview/fast-layout:0.1.0": layout
#import "@preview/cetz:0.5.2"

#let edges = ((0, 1), (1, 2), (2, 3), (3, 0))
#let result = layout(4, edges, algorithm: "stress")
#let xs = result.positions.map(p => p.at(0))
#let ys = result.positions.map(p => p.at(1))
#let min-x = calc.min(..xs)
#let max-x = calc.max(..xs)
#let min-y = calc.min(..ys)
#let max-y = calc.max(..ys)
#let scale = calc.min(
  90mm / calc.max(max-x - min-x, 0.001),
  45mm / calc.max(max-y - min-y, 0.001),
)
#let midpoint = (
  (min-x + max-x) / 2,
  (min-y + max-y) / 2,
)
#let points = result.positions.map(p => (
  p.at(0) - midpoint.at(0),
  p.at(1) - midpoint.at(1),
))

#cetz.canvas(length: scale, {
  import cetz.draw: *
  for edge in edges {
    line(points.at(edge.at(0)), points.at(edge.at(1)))
  }
  for point in points {
    circle(point, radius: 4pt, fill: white)
  }
})
```

The examples choose the smaller available-width and available-height ratio. That preserves angles and relative distances instead of stretching each axis separately.

1.9 Limits and errors

Invalid indices, incompatible options, non-finite values, and malformed trees return errors. Stress and spectral accept at most 4096 nodes. The general request boundary also caps nodes, edges, output coordinates, and encoded input size to prevent unreasonable allocations. These caps do not promise interactive runtime near the limit.

Use spring for large 2D or 3D graphs. For weighted-distance layouts, compare the default 15-update SGD method with explicit majorization on the graphs that matter to your document.